

Static Type Checker - Implementation Overview

(recursive types NOT implemented)

Architecture

The type checker is implemented as a second pass over the AST, separate from evaluation. Every `ASTNode` implements a `typecheck(Environment<ASTType>)` method that returns the inferred `ASTType` or throws `TypeCheckError`. Types are represented as AST nodes themselves (`ASTTInt`, `ASTTBool`, `ASTTArrow`, `ASTTEnum`, `ASTTRef`, `ASTTId`), all implementing the `ASTType` interface which exposes `subTypeOf` supporting the subtypes instead of being limited to equality.

Type Environment and Scoping

A single generic `Environment<T>` is reused for both evaluation and type checking. At each `let` or `type` binding, a new scope is opened with `beginScope()` and closed with `endScope()`, giving lexical scoping for both values and types.

Type Definitions and Alias Resolution

`ASTTypeDef` opens a new type scope and binds each alias name to its structural type. The static method `ASTTypeDef.unfold` resolves a single level of `ASTTId`, if the type is an alias it looks it up in the environment, otherwise it returns it unchanged. When the defined type is an `ASTTEnum`, its constructor tags are also registered directly in the type environment so that `ASTStructTuple` can look them up by name during type checking.

Core Typing Rules

ASTFun extends the environment with the parameter's declared type, typechecks the body, then returns `ASTTArrow(paramType, bodyType)`. Multi-parameter lambdas are curried by the parser into nested `ASTFun` nodes, so each only handles one parameter.

ASTApp unfolds the function's type to get an `ASTTArrow`, unfolds the domain, then checks the argument type is a subtype of the domain via `subTypeOf`. It returns the codomain. Subtyping is used here rather than equality, which is what enables passing a narrower type where a wider one is expected.

ASTLet processes bindings sequentially in a growing local scope. If a binding has a declared type annotation, it unfolds both the inferred and declared types and checks `inferred.subTypeOf(declared)`. If it passes, the declared type (not the inferred one) is stored in the environment, preserving the alias for subsequent bindings. A subtle but important detail: the `typ` variable in the parser is reset to `null` before each binding so that a type annotation on one `let` is never accidentally inherited by the next.

Enum Constructors and Struct Tuples

Enum types (`ASTTEnum`) are maps from tag names to `ASTTStruct` descriptors. Each `ASTTStruct` holds the tag name and an ordered list of field types. When a constructor expression like `#some(2)` is parsed, it becomes an `ASTStructTuple` node. During type checking, `ASTStructTuple` looks up its tag in the type environment to find the `ASTTEnum` it belongs to, retrieves the corresponding `ASTTStruct` (which is only a constructor) to get the expected field types, checks that the number of fields matches exactly, and verifies each field's inferred type is a subtype of the expected type. It returns the full `ASTTEnum` type, not just the constructor's struct, so the result is always the declared union type.

Patterns and Match

Patterns are not typechecked independently (neither were evaluated) - they have no `typecheck` method of their own. Instead, all pattern logic lives in `ASTMatch`.

`ASTMatch` typechecks the scrutinee expression and unfolds its type, which must be an `ASTTEnum`. For each branch it retrieves the `ASTTStruct` for the branch's tag, checks the pattern doesn't bind more variables than the constructor has fields, then opens a local scope and binds each variable to the corresponding (unfolded) field type. Each branch body is then typechecked in this extended scope. All branch result types must be mutually compatible, checked pairwise with `subtypeOf` against the first branch's type.

At runtime, `ASTMatch` evaluates to a `VStructTuple`, iterates over branches until a tag match is found, and binds the runtime field values to the pattern variables in a new scope. Depth subtyping is respected at runtime too: a value may carry more fields than the pattern binds, and the extra fields are simply ignored.

Subtyping

Subtyping is structural and implemented directly in each `ASTType` via `subtypeOf(ASTType t, Environment<ASTType> env)`. The `env` is threaded through so aliases can be resolved at any depth.

LP Project Report Phase2 - Rusty2

Maria João Correia - 86476